

# Proyecto Final: Desarrollo e Implementación de un Sistema de Minería de Datos

## Unidad de Aprendizaje: Minería de Datos

- Elvira Corona Quintero — 2024710056
- Jassiel Luna Cantu — 202410081
- Edna Andrea Serrano Vela — 2024710003
- David Torres Gómez — 2024710012

**Video de Defensa:**

**Repositorio de Github:**

## 1. Introducción

La detección temprana de enfermedades oculares es una de los mayores retos en la oftalmología moderna. Las patologías como la degeneración macular que es asociada a la edad (AMD), la retinopatía diabética (DR) o la neovascularización coroidal (CNV) pueden derivar en pérdidas irreversibles de la visión si es que no se identifican a tiempo. La tomografía de coherencia óptica (OCT) es el estándar de referencia para el análisis retiniano actualmente, ya que produce imágenes transversales de resolución alta de las capas del tejido [1].

El problema que motiva este proyecto surge de la gran cantidad de imágenes OCT que se generan en hospitales y clínicas, lo que analizar cada una manualmente es lento, costoso y propenso a la variabilidad entre observadores. Un sistema que clasifique automáticamente la imagen, y además, recupere casos similares del historial clínico puede ser una herramienta de apoyo muy valioso para un especialista.

Los objetivos del proyecto son los siguientes:

- Desarrollar un clasificador de imágenes OCT de alta precisión empleando transfer learning con MobileNetV2.
- Implementar un módulo CBIR que recupere las cinco imágenes más similares a una consulta usando descriptores GLCM.
- Construir una interfaz accesible para médicos especialistas como herramienta de apoyo

La solución combina técnicas clásicas de extracción de características con arquitecturas modernas de aprendizaje profundo, cumpliendo con los requisitos que se esperan y a la realidad técnica del problema.

## 2. Marco Teórico

### 2.1. Redes Neuronales Convolucionales (CNN)

Una CNN es un tipo de red neuronal diseñada para procesar datos con una estructura de cuadrícula, como las imágenes. Su principal característica son las capas convolucionales, que aplican filtros aprendibles sobre la entrada. La operación de convolución discreta 2D se define como [2]:

$$(I * K)[i, j] = \sum_m \sum_n I[i - m, j - n] \cdot K[m, n] \quad (1)$$

donde  $I$  es la imagen de entrada y  $K$  es el núcleo (kernel) que la red aprende durante el entrenamiento. Después de varias capas convolucionales intercaladas con capas de agrupamiento (pooling), las características extraídas pasan a capas densas que realizan la clasificación final.

### 2.2. Transfer Learning y MobileNet V2

Entrenar una CNN desde cero requiere millones de imágenes etiquetadas y tiempos de cómputo elevados. El transfer learning consiste en tomar una red ya entrenada en un conjunto de datos grande (en este caso, ImageNet) y adaptar sus capas superiores al problema específico [3].

MobileNetV2 es una arquitectura eficiente desarrollada por Google que emplea bloques de cuello de botella invertido (*inverted residual blocks*) y convoluciones separables en profundidad (*depthwise separable convolutions*). Estas operaciones reducen significativamente el número de parámetros respecto a arquitecturas clásicas sin sacrificar precisión, lo que la hace adecuada para entornos con recursos computacionales limitados [4].

El fine-tuning extiende esta idea descongelando gradualmente las últimas capas de la base pre-entrenada y reentrenándolas con una tasa de aprendizaje muy pequeñas, permitiendo que el modelo ajuste a sus filtros de bajo nivel a las características de dominio médico.

### 2.3. Matriz de Co-ocurrencia de Niveles de Gris (GLCM)

La GLCM (*Gray-Level Co-occurrence Matrix*) es una herramienta clásica para describir la textura de una imagen en escala de grises [5]. Se define como una matriz  $G$  de tamaño  $L \times L$ , donde cada entrada  $G[i, j]$  contabiliza cuántas veces el nivel de intensidad  $i$  aparece a cierta distancia y ángulo del nivel  $j$ .

De esta matriz se extraen descriptores estadísticos que capturan propiedades de la textura:

$$\text{Contraste} = \sum_{i,j} (i - j)^2 \cdot G[i, j] \quad (2)$$

$$\text{Homogeneidad} = \sum_{i,j} \frac{G[i, j]}{1 + |i - j|} \quad (3)$$

$$\text{Energía} = \sum_{i,j} G[i, j]^2 \quad (4)$$

$$\text{Correlación} = \sum_{i,j} \frac{(i - \mu_i)(j - \mu_j) G[i, j]}{\sigma_i \sigma_j} \quad (5)$$

En nuestro sistema se extraen cinco propiedades por imagen (contraste, homogeneidad, energía, correlación y ASM), formando un vector de características de 5 dimensiones que sirve como descriptor compacto para el módulo CBIR.

## 2.4. Recuperación de Imágenes por Contenido (CBIR)

Un sistema CBIR (*Content-Based Image Retrieval*) permite encontrar imágenes visualmente similares a una consulta baándose en el contenido visual y no en metadatos [6]. El proceso general es:

1. Extraer un vector de características (descriptor) de la imagen consultada.
2. Calcular la distancia entre ese vector y los descriptores de todas las imágenes en la base de datos.
3. Devolver las  $k$  imágenes con menor distancia.

Las métricas de distancia implementadas en el sistema son:

$$d_{\text{Eucl}}(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2} \quad (6)$$

$$d_{\text{Manhattan}}(p, q) = \sum_{i=1}^n |p_i - q_i| \quad (7)$$

$$d_{\text{coseno}}(p, q) = 1 - \frac{p \cdot q}{\|p\| \cdot \|q\|} \quad (8)$$

## 3. Desarrollo (Metodología CRISP-DM)

### 3.1. Compresión del Negocio

El proyecto simula el contexto de un hospital oftalmológico que maneja un volumen creciente de imágenes OCT. Las necesidades identificadas son:

- Un sistema que sugiera un diagnóstico automático acompañado de un nivel de confianza cuantificado.
- Un mecanismo de soporte que muestre casos previos similares para que el médico pueda contrastarlos con su criterio clínico

Las ocho clases de patología consideradas son: **AMD** (degeneración macular asociada a la edad), **CNV** (neovascularización coroidal), **CSR** (coriorretinopatía serosa central), **DME** (edema macular diabético), **DR** (retinopatía diabética), **DRUSEN**, **MH** (agujero macular) y **NORMAL**.

### 3.2. Compresión de los datos (EDA)

El dataset utilizado fue el **RetinalOCT-C8** [7], disponible en la plataforma Kaggle, que contiene imágenes OCT de retina organizadas en tres particiones. La descarga se realizó directamente con la API de KaggleHub.

```

1 import os
2 import pandas as pd
3 import kagglehub
4
5 # Descargar dataset desde Kaggle
6 dataset_path = kagglehub.dataset_download("obulisainaren/retinal-oct-c8"
7 )
8 base_path = os.path.join(dataset_path, "RetinalOCT_Dataset/"
9 RetinalOCT_Dataset")
10
11 def crear_dataframe(split_path):
12     datos = []
13     for clase in os.listdir(split_path):
14         clase_path = os.path.join(split_path, clase)
15         if os.path.isdir(clase_path):
16             for archivo in os.listdir(clase_path):
17                 datos.append({"filepath": os.path.join(clase_path,
18                 archivo),
19                             "label": clase})
20     return pd.DataFrame(datos)
21
22 df_train = crear_dataframe(os.path.join(base_path, "train"))
23 df_val = crear_dataframe(os.path.join(base_path, "val"))
24 df_test = crear_dataframe(os.path.join(base_path, "test"))
25
26 print("Train:", df_train.shape) # (N_train, 2)
27 print("Val:", df_val.shape)
28 print("Test:", df_test.shape)

```

Listing 1: Descarga del dataset y construcción de DataFrames.

Cuadro 1: Distribución del dataset RetinalOCT-C8

| Partición             | Imágenes | Clases |
|-----------------------|----------|--------|
| Entrenamiento (train) | >10,000  | 8      |
| Validación (val)      | ~1,000   | 8      |
| Prueba (test)         | ~1,000   | 8      |

Durante la exploración visual se obtuvo una muestra aleatoria de cada clase 1, Las imágenes OCT presentan características visuales distintas según la patología. Esta variabilidad visual sugiere que tanto los descriptores de textura como las características aprendidas por la CNN son relevantes para la tarea.

### 3.3. Preprocesamiento y Pipeline

Se realizó un preprocesamiento inicial de las imágenes para la extracción de características de textura mediante GLCM y para la construcción del sistema CBIR. Posteriormente, durante

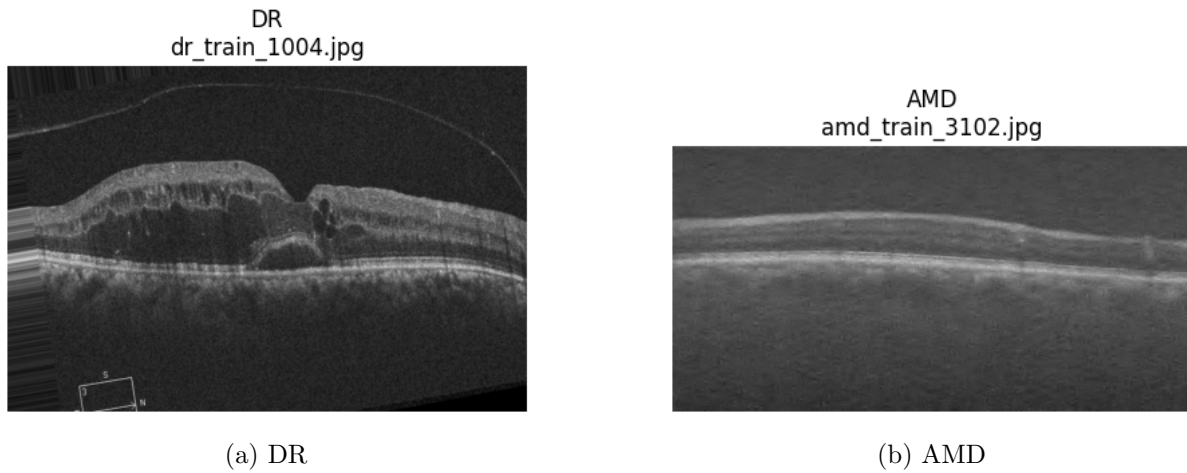


Figura 1: OCT de muestra del dataset

el entrenamiento de la CNN MobileNetV2, las imágenes fueron cargadas nuevamente mediante ImageDataGenerator, el cual aplicó automáticamente redimensionamiento, normalización y aumento de datos.

```

1 from skimage.feature import graycomatrix, graycoprops
2
3 def extraer_glcmm(img):
4     gray = cv.cvtColor(img, cv.COLOR_RGB2GRAY)
5     glcm = graycomatrix(
6         gray,
7         distances=[1],
8         angles=[0],
9         levels=256,
10        symmetric=True,
11        normed=True
12    )
13
14    contraste = graycoprops(glcm, "contrast")[0, 0]
15    homogeneidad = graycoprops(glcm, "homogeneity")[0, 0]
16    energia = graycoprops(glcm, "energy")[0, 0]
17    correlacion = graycoprops(glcm, "correlation")[0, 0]
18    asm = graycoprops(glcm, "ASM")[0, 0] # Segundo momento angular
19
20    return np.array([
21        contraste,
22        homogeneidad,
23        energia,
24        correlacion,
25        asm
26    ])

```

Listing 2: Preprocesamiento para GLCM.

El preprocesamiento se diseñó en dos líneas paralelas, una para cada módulo del sistema.

### 3.3.1. Pipeline para la CNN (con aumento de datos)

Las imágenes se redimensionaron a  $224 \times 224$  píxeles (requisito para MobileNetV2) y los valores de píxel se normalizaron al rango  $[0, 1]$ . Para el conjunto de entrenamiento se aplicó *data*

*augmentation* para reducir el sobreajuste.

```

1 from tensorflow.keras.preprocessing.image import ImageDataGenerator
2
3 # Augmentacion solo en entrenamiento
4 train_gen = ImageDataGenerator(
5     rescale=1./255,
6     rotation_range=10,          # Rotaciones leves (imagenes medicas)
7     width_shift_range=0.05,
8     height_shift_range=0.05,
9     horizontal_flip=True
10 )
11 val_test_gen = ImageDataGenerator(rescale=1./255) # Solo normalizar
12
13 train_data = train_gen.flow_from_directory(
14     os.path.join(base_path, "train"),
15     target_size=(224, 224), batch_size=32, class_mode="categorical"
16 )
17 val_data = val_test_gen.flow_from_directory(
18     os.path.join(base_path, "val"),
19     target_size=(224, 224), batch_size=32, class_mode="categorical"
20 )
21 test_data = val_test_gen.flow_from_directory(
22     os.path.join(base_path, "test"),
23     target_size=(224, 224), batch_size=32, class_mode="categorical",
24     shuffle=False # Importante para evaluar correctamente
25 )

```

Listing 3: Generadores de datos con aumento para entrenamiento.

### 3.3.2. Pipeline para CBIR (descriptores GLCM)

Para el módulo de recuperación, cada imagen se convirtió a escala de grises y se calculó su GLCM con distancia 1, ángulo 0°, 256 niveles, modo simétrico y normalizada. Se extraen cinco propiedades estadísticas formando un vector de 5 dimensiones por imagen.

```

1 import numpy as np
2 import cv2 as cv
3 from skimage.feature import graycomatrix, graycoprops
4
5 def extraer_glcmm(img):
6     gray = cv.cvtColor(img, cv.COLOR_RGB2GRAY)
7     glcm = graycomatrix(gray, distances=[1], angles=[0],
8                         levels=256, symmetric=True, normed=True)
9     return np.array([
10         graycoprops(glcm, "contrast")[0, 0], # Variacion de
11         graycoprops(glcm, "homogeneity")[0, 0], # Uniformidad de textura
12         graycoprops(glcm, "energy")[0, 0], # Uniformidad de
13         graycoprops(glcm, "correlation")[0, 0], # Relacion lineal entre
14         graycoprops(glcm, "ASM")[0, 0] # Segundo momento
15         angular
16     ])

```

Listing 4: Extracción de descriptores GLCM por imagen.

Evaluamos combinar descriptores HSV con GLCM, pero se optó por GLCM solo para el módulo CBIR. Las imágenes OCT son inherentemente en escala de grises y la textura es su característica más discriminativa. Los histogramas no aportarían información relevante en este dominio.

### 3.4. Modelado

#### 3.4.1. Fase 1: Transfer Learning con base congelada

Se cargó MobileNetV2 preentrenada con pesos de ImageNet y se congeló la totalidad de sus capas. Además, se construyó un clasificador personalizado.

```

1 from tensorflow.keras.applications import MobileNetV2
2 from tensorflow.keras import layers, models
3
4 base_model = MobileNetV2(
5     input_shape=(224, 224, 3),
6     include_top=False,          # Sin la capa de clasificacion original
7     weights="imagenet"         # Pesos preentrenados en ImageNet
8 )
9 base_model.trainable = False   # Congelamos la base
10
11 model = models.Sequential([
12     base_model,
13     layers.GlobalAveragePooling2D(), # Reduce el mapa de
    características
14     layers.Dense(128, activation="relu"),
15     layers.Dropout(0.3),          # Regularizacion
16     layers.Dense(8, activation="softmax") # 8 clases de salida
17 ])
18
19 model.compile(optimizer="adam",
20               loss="categorical_crossentropy",
21               metrics=["accuracy"])

```

Listing 5: Construcción del modelo con MobileNetV2 como base.

#### 3.4.2. Fase 2: Fine-Tuning

En la segunda fase se descongelaron las últimas 30 capas de la base y se reentrenó todo el sistema con una tasa de aprendizaje muy reducida ( $1 \times 10^{-5}$ ). Se utilizaron *callbacks* de EarlyStopping y ModelCheckpoint para guardar el mejor modelo y evitar sobreajuste.

```

1 import tensorflow as tf
2 from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
3
4 callbacks = [
5     EarlyStopping(patience=5, restore_best_weights=True),
6     ModelCheckpoint("mejor_modelo.keras", save_best_only=True)
7 ]
8
9 base_model.trainable = True
10 for layer in base_model.layers[:-30]:
11     layer.trainable = False # Solo las ultimas 30 capas se ajustan
12
13 model.compile(

```

```

14     optimizer=tf.keras.optimizers.Adam(1e-5),
15     loss="categorical_crossentropy",
16     metrics=["accuracy"]
17 )
18
19 historia_ft = model.fit(
20     train_data, validation_data=val_data,
21     epochs=10, callbacks=callbacks
22 )

```

Listing 6: Fine-tuning con capas descongeladas y callbacks.

### 3.4.3. Módulo CBIR

Una vez entrenado el clasificador, se construyó la base de datos de recuperación calculando el descriptor GLCM de cada imagen del conjunto de entrenamiento. La función principal del módulo integra clasificación y recuperación en un solo paso.

```

1 from scipy.spatial.distance import euclidean, cityblock, cosine
2
3 def buscar_similares(img_path, top_k=5, metrica="euclidean"):
4     # --- Clasificación con la CNN ---
5     img_orig = cv.imread(img_path)
6     img_224 = cv.resize(img_orig, (224, 224))
7     img_rgb = cv.cvtColor(img_224, cv.COLOR_BGR2RGB)
8     img_input = np.expand_dims(img_rgb / 255.0, axis=0)
9
10    pred_probs = model.predict(img_input, verbose=0)[0]
11    pred_clase = list(train_data.class_indices.keys())[np.argmax(
12    pred_probs)]
13    confianza = pred_probs.max() * 100
14
15    # --- Descriptor GLCM de la imagen consulta ---
16    query_desc = extraer_glcmm(img_rgb)
17
18    # --- Calcular distancias contra la base CBIR ---
19    dist_fn = {"euclidean": euclidean, "manhattan": cityblock,
20    "cosine": cosine}[metrica]
21    distancias = np.array([dist_fn(query_desc, d) for d in
22    db_descriptores])
23    indices_top = np.argsort(distancias)[:top_k]
24
25    return pred_clase, confianza, indices_top, distancias

```

Listing 7: Función principal del sistema CBIR.

## 3.5. Evaluación

### 3.5.1. Métricas de Clasificación

El modelo se evaluó sobre el conjunto de prueba, que no fue utilizado durante el entrenamiento. Se generaron precisión, recall y F1-Score por Clase, así como la matriz de confusión.



```

1 from sklearn.metrics import classification_report, confusion_matrix
2
3 y_pred_probs = model.predict(test_data)
4 y_pred       = np.argmax(y_pred_probs, axis=1)
5 y_true       = test_data.classes
6 class_names  = list(test_data.class_indices.keys())
7
8 print(classification_report(y_true, y_pred, target_names=class_names))

```

Listing 8: Cálculo de métricas y matriz de confusión.

Cuadro 2: Resumen de métricas del modelo fine-tuneado por clase

| Clase            | Precisión   | Recall      | F1-Score    |
|------------------|-------------|-------------|-------------|
| AMD              | 1.00        | 1.00        | 1.00        |
| CNV              | 0.89        | 0.82        | 0.85        |
| CSR              | 0.97        | 1.00        | 0.99        |
| DME              | 0.95        | 0.82        | 0.85        |
| DR               | 1.00        | 0.98        | 0.99        |
| DRUSEN           | 0.77        | 0.84        | 0.80        |
| MH               | 1.00        | 1.00        | 1.00        |
| NORMAL           | 0.85        | 0.94        | 0.89        |
| <b>Macro avg</b> | <b>0.93</b> | <b>0.92</b> | <b>0.92</b> |

### 3.5.2. Análisis de la Matriz de confusión

La matriz de confusión permite identificar entre qué clases existe mayor ambigüedad 4. Se observa que el modelo basado en MobileNetV2 obtuvo un desempeño general elevado, ya que la mayoría de las predicciones se concentran sobre la diagonal principal, indicando una correcta clasificación de las imágenes. Las clases AMD, CSR y MH alcanzaron una clasificación perfecta, mientras que los errores se concentraron principalmente entre CNV, DRUSEN, DME y NORMAL, categorías que presentan características visuales más similares y representan los principales desafíos para el modelo.

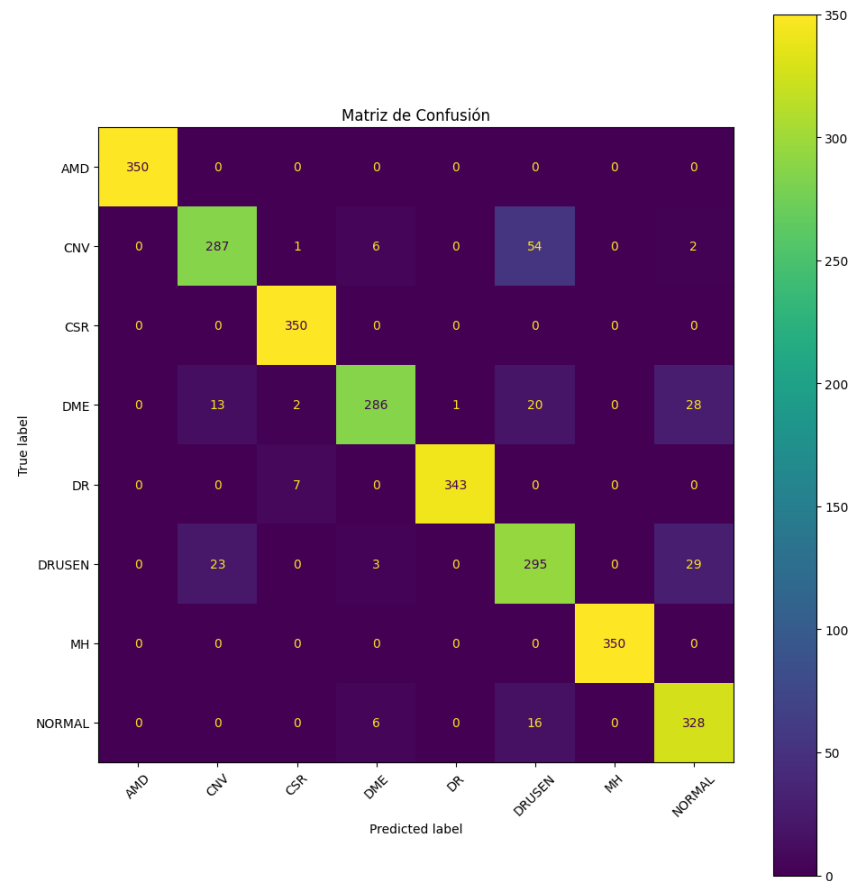


Figura 2: Matriz de confusión

### 3.5.3. Curvas de aprendizaje (Fine-Tuning)

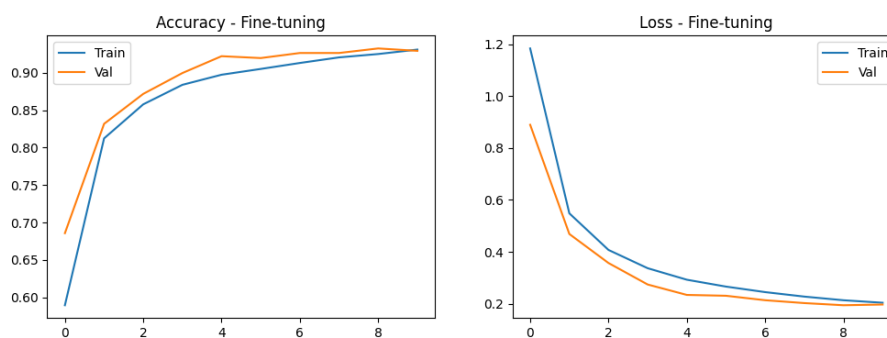


Figura 3: Gráficas de las Curvas de Aprendizaje

El modelo converge de forma estable: la exactitud de validación sube desde 68.6 % en la primera época hasta 93.3 % en la época 9 (mejor punto según val\_loss), sin signos de sobreajuste severo, ya que la brecha entre accuracy de entrenamiento y validación se mantiene pequeña a lo largo de todo el proceso.

La Tabla 3 resume los valores reportados en el notebook para cada época:

Cuadro 3: Historial de entrenamiento – fine-tuning (10 épocas)

| Épica | Acc. Train | Loss Train | Acc. Val | Loss Val |
|-------|------------|------------|----------|----------|
| 1     | 0.5895     | 1.1839     | 0.6857   | 0.8897   |
| 2     | 0.8123     | 0.5483     | 0.8318   | 0.4687   |
| 3     | 0.8578     | 0.4072     | 0.8718   | 0.3568   |
| 4     | 0.8839     | 0.3368     | 0.8996   | 0.2740   |
| 5     | 0.8974     | 0.2924     | 0.9221   | 0.2339   |
| 6     | 0.9051     | 0.2660     | 0.9196   | 0.2306   |
| 7     | 0.9131     | 0.2447     | 0.9264   | 0.2134   |
| 8     | 0.9205     | 0.2274     | 0.9264   | 0.2024   |
| 9     | 0.9251     | 0.2133     | 0.9325   | 0.1941   |
| 10    | 0.9310     | 0.2037     | 0.9293   | 0.1968   |

### 3.5.4. Resultados del Módulo CBIR

El módulo CBIR recupera las cinco imágenes más similares a una consulta usando los descriptores GLCM .

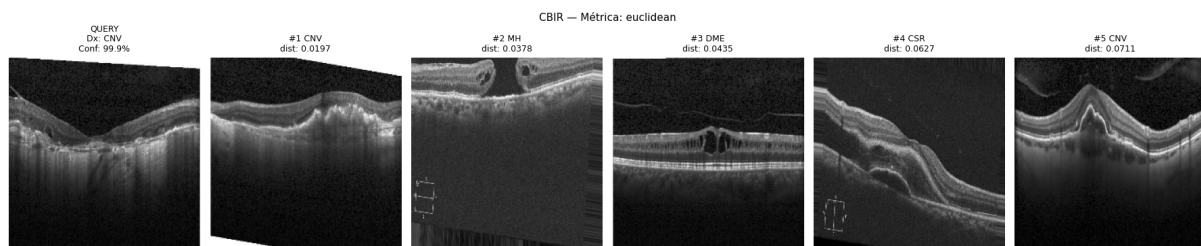


Figura 4: Ejemplo de recuperación CBIR: imagen consulta (izquierda) y las 5 imágenes más similares recuperadas por distancia GLCM (derecha).

Las imágenes recuperadas pertenecen a la misma clase que la consulta en la mayoría de los casos, lo que prueba que los descriptores GLCM capturan características discriminativas relevantes de la textura retiniana.

## 4. Interfaz

La interfaz fue implementada usando Tkinter, buscando que fuera simple e intuitiva de usar (esto pensando en una situación rela), debido a su facilidad para construirla. Esto nos permite visualizar en tiempo real los resultados del modelo y análisis que se hagan por la imagen cargada. Al cargarse una imagen por el usuario, se redimensiona a 224x224 píxeles, con esto se asegura la compatibilidad con el modelo entrenado. Una vez que la imagen es procesada, se envía al modelo, y este retorna un vector con las probabilidades que corresponden a las clases de las enfermedades, donde se procesa para obtener la clase con la mayor probabilidad

### 4.1. Resultados

Como resultados la interfaz organiza la información en dos columnas principales:

- En la primera se muestra la imagen que el usuario subió.

- En la segunda se muestran los resultados del diagnóstico.

En una última sección se visualizan las imágenes que más parecido llega a tener con la imagen original, permitiendo una comparación visual entre el caso que se analiza y los ejemplos del conjunto de datos.

## 5. Conclusión

El proyecto permitió construir un sistema funcional que integra clasificación automática de imágenes OCT con recuperación de casos similares, atendiendo una necesidad real en oftalmología clínica. Los resultados obtenidos son sólidos: exactitud global del 92 %, con F1-Score macro de 0.92, superando ampliamente cualquier línea base trivial. Durante el desarrollo enfrentamos varios retos concretos:

**Heterogeneidad del dataset.** Las imágenes del RetinalOCT-C8 presentan variaciones en iluminación y contraste que dificultaron la extracción de descriptores GLCM consistentes. El data augmentation y la normalización ayudaron a mitigar este problema para el clasificador CNN.

**Elección de la representación para CBIR.** Evaluamos combinar descriptores HSV con GLCM, pero la naturaleza en escala de grises de las imágenes OCT hace que el color no aporte información discriminativa. Optar por GLCM puro simplificó el pipeline sin perder calidad en la recuperación.

**Balance entre transfer learning y fine-tuning.** Congelar toda la base de MobileNetV2 durante una primera fase y descongelar solo las últimas 30 capas en el fine-tuning resultó en una estrategia estable. El historial de entrenamiento muestra que la convergencia es progresiva y sin sobreajuste significativo, con la mejor exactitud de validación (93.25 %) alcanzada en la época 9.

En cuanto a los hallazgos semánticos, la GLCM captura diferencias de textura coherentes con la naturaleza de las patologías: los tejidos con fluido acumulado muestran mayor contraste y menor homogeneidad respecto a retina normales. Las confusiones más frecuentes (DRUSEN con NORMAL y AMD, CNV con DRUSEN) corresponden a clases con similitudes patofisiológicas reales, lo que valida la sensatez del comportamiento del modelo.

## Referencias

- [1] D. Huang, E. A. Swanson, C. P. Lin, J. S. Schuman *et al.*, “Optical coherence tomography,” *Science*, vol. 254, no. 5035, pp. 1178–1181, 1991. [En línea]. Disponible: <https://www.science.org/doi/10.1126/science.1957169>
- [2] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [En línea]. Disponible: <https://ieeexplore.ieee.org/document/726791>
- [3] S. J. Pan y Q. Yang, “A survey on transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2009. [En línea]. Disponible: <https://ieeexplore.ieee.org/document/5288526>
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov y L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” en *Proc. IEEE Conf. Computer Vision and Pattern Recogni-*

- tion (CVPR), 2018, pp. 4510–4520. [En línea]. Disponible: <https://arxiv.org/abs/1801.04381>
- [5] R. M. Haralick, K. Shanmugam e I. H. Dinstein, “Textural features for image classification,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. [En línea]. Disponible: <https://ieeexplore.ieee.org/document/4309314>
- [6] A. W. M. Smeulders, M. Worring, S. Santini, A. Gupta y R. Jain, “Content-based image retrieval at the end of the early years,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 22, no. 12, pp. 1349–1380, 2000. [En línea]. Disponible: <https://ieeexplore.ieee.org/document/895972>
- [7] Obulisainaren, “Retinal OCT C8 Dataset,” Kaggle, 2023. [En línea]. Disponible: <https://www.kaggle.com/datasets/obulisainaren/retinal-oct-c8>
- [8] S. van der Walt *et al.*, “scikit-image: Image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014. [En línea]. Disponible: <https://doi.org/10.7717/peerj.453>
- [9] M. Abadi *et al.*, “TensorFlow: A system for large-scale machine learning,” en *12th USENIX Symp. Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 265–283. [En línea]. Disponible: <https://www.tensorflow.org>